# Reviewer Pack — Resolution Width Lower Bound for Tseitin Formulas on Expande…

Entry 4c6e3be1d344 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 07:02:16 UTC

## Resolution Width Lower Bound for Tseitin Formulas on Expander Graphs

**Entry ID**: `4c6e3be1d344` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 07:02:16 UTC

### 1. Conjecture

**Field A** (mathematical branch): GRAPH_EXPANSION **Field B** (complexity object): RESOLUTION_WIDTH

**Statement**:

> For any expander graph G with expansion Φ, the resolution width of the Tseitin formula T(G, ω) is Ω(1/Φ), where ω is a balanced assignment.

**Rationale (proposer's reasoning)**:

> Expander graphs have strong connectivity properties, which may necessitate wider proofs in resolution systems. The conjecture links the expansion parameter to the required proof width, suggesting that graphs with higher expansion require wider proofs, thus increasing the complexity.

**Taxonomy category**: `PROOF_COMPLEXITY_TSEITIN` (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `6e51b348f80e73f3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.95 | SAFE | SAFE |
| NATURAL_PROOFS | SAFE | 0.95 | SAFE | SAFE |
| ALGEBRIZATION | SAFE | 0.95 | SAFE | SAFE |

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| KARP_LIPTON | SAFE | 0.95 | UNCERTAIN | SAFE |

## 4. Novelty audit

**Verdict**: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_expander_graph(n, Φ):
        # Simple expander graph generator (adjacency list)
        G = {i: [] for i in range(n)}
        for v in range(n):
            for u in range(v + 1, n):
                if len(G[v]) < Φ and len(G[u]) < Φ:
                    G[v].append(u)
                    G[u].append(v)
        return G

    def tseitin_formula(G, ω):
        literals = {}
        clauses = []

        def get_literal(v, i):
            var_id = literals.get((v, i), None)
            if var_id is None:
                var_id = len(literals)
                literals[(v, i)] = var_id
            return var_id

        for v in G:
            literal_v0 = get_literal(v, 0)
            literal_v1 = get_literal(v, 1)
            clauses.append([literal_v0, -literal_v1])
            clauses.append([-literal_v0, literal_v1])

        for u in range(len(G)):
            for v in range(u + 1, len(G)):
                if u in G[v]:
                    literal_u0 = get_literal(u, 0)
                    literal_v0 = get_literal(v, 0)
                    literal_u1 = get_literal(u, 1)
```

```
                    literal_v1 = get_literal(v, 1)
                    clauses.append([literal_u0, literal_v0, -literal_u1, -literal_v1])

        return literals, clauses

    def resolution_width(clauses):
        # Simple resolution width computation
        max_width = 0
        seen = set()
        queue = list(clauses)

        while queue:
            clause = queue.pop(0)
            if len(clause) > max_width:
                max_width = len(clause)

            for literal in clause:
                opposite_literal = -literal
                if opposite_literal in seen:
                    continue

                new_clauses = []
                for other_clause in clauses:
                    if opposite_literal in other_clause:
                        new_clause = [l for l in other_clause if l != opposite_literal]
                        if new_clause not in new_clauses:
                            new_clauses.append(new_clause)

                queue.extend(new_clauses)
                seen.add(opposite_literal)

        return max_width

    n = 30
    Φ = random.randint(2, n - 1)  # Expansion parameter
    expander_graph = generate_expander_graph(n, Φ)
    ω = {v: random.choice([True, False]) for v in range(n)}

    literals, clauses = tseitin_formula(expander_graph, ω)
    width = resolution_width(clauses)

    return {
        "metric_name": "resolution_width",
        "metric_value": width,
        "instances_tested": 1,
        "conjecture_holds": width >= 1 / Φ,
        "counterexample": "" if width >= 1 / Φ else f"Graph with n={n}, Φ={Φ} did not meet the low
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    std_width = math.sqrt(sum((r["metric_value"] - mean_width) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph did not meet the lower bound\" first_fail
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
value': 4, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 4, 'instances_tested': 1, 'conjecture_hol
RESULT: SUPPORTED mean=4.0 std=0.0 support_fraction=1.0
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> Single test case meets support fraction but critic challenges validity due to insufficient diversity. Pre-registered criterion requires >=80% seeds, which is technically met but lacks statistical confidence. | next: Test with multiple expander graphs and varying expansion parameters to validate across diverse instances

## 11. Audit log (LLM calls)

**Total LLM calls**: 11

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 70908 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 71946 |
| 3 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 29575 |
| 4 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 24093 |
| 5 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 21085 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 23689 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 10771 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11759 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11659 |
| 10 | critic | ollama_remote | qwen3:8b | 0 | 0 | 32886 |
| 11 | judge | ollama_remote | qwen3:8b | 0 | 0 | 16725 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 325095 ms total latency. Provider mix: {'ollama_remote': 11}

*(full prompt+response transcripts available in `research/audit/4c6e3be1d344.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4c6e3be1d344.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/4c6e3be1d344.tar.gz` (if generated)